第 16 节 实操任务 1：上线前 Checklist 验证

> 学习目标：8 项 Checklist 全部通过
> 前置要求：13/14/15 节已跑通（Docker 化是加餐，无 Docker 也能查这些项）

背景

Checklist 不是形式主义，每一项对应一个可能导致线上事故的风险点。跳过任何一项就是在赌运气。

以下验证命令可以手动执行，也可以让 **OpenCode**、**Claude Code** 等 AI 编程工具帮你逐项检查。

检查 1：API Keys 环境变量

```
# .env 文件存在且权限正确
ls -la .env
# 期望：-rw------- (600) 或 -rw-r----- (640)

# .env 不在 Git 中
git ls-files .env
# 期望：无输出（说明没有被跟踪）

# .env.example 存在
cat .env.example
```

检查项	状态
.env 文件存在	[ ]
.env 已加入 .gitignore	[ ]
.env.example 包含所有必需变量	[ ]

检查 2：权限策略

```
# 检查全部 2 个 Skill 的 allowed-tools（daily-digest / top-rated）
grep -r "allowed-tools" -A 3 \
  ~/ai-knowledge-base/v4-production/openclaw/skills/

# 期望:每个 Skill 只有 Read(messaging profile 不识别 Glob/Grep,
# 加了反而会让 Bot 触发 exec 报错)
```

```

# 检查全局 OpenClaw 工具策略
openclaw config get tools.alsoAllow
# 期望: ["read"]

# 全 workspace 扫一遍 · 任何 SKILL.md / AGENTS.md 都不应再提 Glob / Grep / e
grep -rn -i "glob\|grep\|exec" \
  ~/ai-knowledge-base/v4-production/openclaw/skills/*/SKILL.md \
  ~/ai-knowledge-base/v4-production/openclaw/AGENTS.md \
  ~/ai-knowledge-base/v4-production/AGENTS.md \
  | grep -v "^[^:]*:\s*$" || echo "✓ 干净"
# 期望: ✓ 干净（或只命中无害的注释/示例文本）
```

检查项	状态
所有 Skill 的 allowed-tools 只有 Read	[ ]
Skill 正文 / AGENTS.md 不再提 Glob/Grep/exec	[ ]
全局 tools.alsoAllow = `["read"]` (不开 bash/write)	[ ]
没有 Skill 写入 knowledge/ (写权限集中在 pipeline)	[ ]

检查 3：备份策略

```
# knowledge 目录有数据
ls knowledge/articles/

# Docker 镜像有版本标签
docker images | grep ai-knowledge-base

# Git 仓库有远程
git remote -v
```

检查项	状态
knowledge/ 目录有数据文件	[ ]
Docker 镜像打了版本标签	[ ]
Git 仓库推送到 GitHub	[ ]

检查 4：日志轮转

```
# docker-compose.yml 中日志配置
grep -A 4 "logging" docker-compose.yml
```

检查项	状态
max-size 设为 10m	[ ]
max-file 设为 3	[ ]
logs/ 目录不包含敏感信息	[ ]

检查 5：成本预算

```
# CostGuard 配置
grep -r "daily_budget\|per_pipeline_budget\|max_calls" *.py pipeline/*.p
```

检查项	状态
CostGuard daily_budget 已设置	[ ]
熔断器 max_calls 已设置	[ ]
月度成本预估合理	[ ]

检查 6：版本固定

```
# Python 依赖有版本号
cat requirements.txt

# Docker 基础镜像指定版本
grep "FROM" Dockerfile
```

检查项	状态
requirements.txt 包含版本号	[ ]
Docker 基础镜像 = python:3.12-slim（非 latest）	[ ]

检查 7：测试通道

```
# 手动跑一次 Pipeline
python3 pipeline/pipeline.py 2>/dev/null || echo "Pipeline 需要根据你的实现

# 运行单元测试
python3 -m pytest tests/ 2>/dev/null || echo "确保 tests/ 目录存在"
```

检查项	状态
手动跑一次完整管线	[ ]
单元测试通过	[ ]
Telegram Bot 发送测试消息	[ ]

检查 8：回滚方案

```
# 确认知道如何回滚
echo "回滚步骤："
echo "1. docker compose down"
echo "2. 修改 docker-compose.yml 中 image 为上个版本标签"
echo "3. docker compose up -d"
```

检查项	状态
知道如何回滚 Docker 镜像	[ ]
知道如何恢复数据备份	[ ]

检查 9：OpenClaw Bot 接管 Telegram（V4 新增）

```
# daemon 在跑
ss -tlnp | grep 18789 >/dev/null && echo "[OK] daemon 监听 18789" || echo

# 默认模型不是 gpt-5.5 占位符
MODEL=$(openclaw capability model auth status 2>/dev/null | python3 -c "
[ "$MODEL" != "openai/gpt-5.5" ] && echo "[OK] 模型已切到 $MODEL" || echo

# 模型可调
openclaw capability model run --model "$MODEL" --prompt "ping" 2>&1 | gr
  && echo "[OK] 模型可调用" || echo "[!!] 401 / 网络 / 配置错"

# workspace 切到 v4
[ "$(openclaw config get agents.defaults.workspace)" = "/home/$USER/ai-k
  && echo "[OK] workspace 已切到 v4" || echo "[!!] workspace 不对"
```

检查项	状态
OpenClaw daemon 监听 18789	[ ]
默认模型 ≠ openai/gpt-5.5（已切 DeepSeek）	[ ]
模型可调用（capability run 返回 outputs: 1）	[ ]
workspace = v4-production/openclaw	[ ]

检查 10：GitHub Actions 自动采集（V4 新增）

```
# fork 后必须配的 secret
gh secret list 2>&1 | grep DEEPSEEK_API_KEY \
  && echo "[OK] DEEPSEEK_API_KEY 已配" || echo "[!!] 去 Settings → Secret

# 最近 workflow 跑成功了吗
gh run list --workflow daily-collect-v4.yml --limit 3 2>&1 | head -5
```

检查项	状态
DEEPSEEK_API_KEY secret 已配	[ ]
daily-collect-v4 最近 24h 内有 success	[ ]
知识库每天有新文章入库（git log 看 chore(v4) commit）	[ ]

汇总

```
上线前 Checklist — V4 知识库系统（10 项完整版）
═══════════════════════════════════════════
[ ] 1. API Keys 环境变量
[ ] 2. 权限策略（OpenClaw Skill / tools.alsoAllow）
[ ] 3. 备份策略
[ ] 4. 日志轮转
[ ] 5. 成本预算
[ ] 6. 版本固定
[ ] 7. 测试通道
[ ] 8. 回滚方案
[ ] 9. OpenClaw Bot 接管 Telegram（V4 新增）
[ ] 10. GitHub Actions 自动采集（V4 新增）
```

10 项全绿才算具备上线条件。任意 [!!] 必须修复。

完成！ Checklist 全部通过，系统具备上线条件。进入实操 2 提交毕业项目。